Questions

- Why is the IO controller separated from the CPU?
  - More space needed with diminishing returns
- Why is device tree not burned into the hardware?
  - Device Tree is not OS agnostic.
- Call Up vs Teletype?
-

The Halt Stub

```
.section ".text.boot"
.global _start

_start:
    wfe
    b _start
```

- **start.s**: The smallest valid AArch64 execution context. We'll cover ARM in depth soon!
- **wfe (Wait For Event):** Instruction to physically gate the ALU clock. Suspends instruction execution, dropping the core into a low-power static state.
- **b _start (Branch Unconditional):** Modifies the Program Counter (PC) to the address of **_start**. Failsafe against spurious hardware wakeups.

Cross-Compilation

- **Native Compilation** is compiling a C program on your OS to run on your OS. The compiler uses your hardware's ISA and links your OS libraries.
- **Cross-Compilation** is running a compiler on your OS that generates raw machine code for a completely different architecture and environment (in our case, an environment with no operating system).
- **The Toolchain** (aarch64-elf or aarch64-none-elf):
  - aarch64: The target architecture (64-bit ARM).
  - none: The target operating system (Bare-metal / No OS).
  - elf: The target file format.

Build Pipeline

1. **The Assembler (as): Text→Object (.o)**
    a. Translates human mnemonics (wfe) into 32-bit binary (0xd503205f).
    b. Leaves all memory addresses blank (0x0000).
2. **The Linker (ld): Objects→Executable (.elf)**
    a. Uses a Linker Script (.ld) as a blueprint.
    b. Anchors the blank code to physical RAM addresses (e.g., 0x80000).
    c. Produces an ELF (Executable and Linkable Format). Contains code + metadata (headers, symbols).
3. **Objcopy (objcopy): ELF→Raw Binary (.img)**

Firmware Logs with an SD Card

```
4.24 Boot mode: SD (01) order f4
4.72 Trying partition: 0
4.75 type: 32 lba: 2048 'BSD  4.4' ' BOOT       ' clusters 1948483 (64)
4.88 FAT32 clusters 1948483
5.25 Read config.txt bytes      154 hnd 0x9d2
5.50 Read bcm2712-rpi-5-b.dtb bytes    78231 hnd 0xc5
```

A filtered version is pasted here.

Some highlights

- The GPU successfully identifies the SD card and mounts the FAT32 partition (Type 32).
- The firmware locates and parses our configuration file.
- The 78-kilobyte Device Tree (.dtb) hardware map is loaded into RAM.
- The firmware locates kernel8.img, drops it at physical address 0x80000, and releases the ARM Cortex-A76 cores from their reset state.
- The ARM CPU wakes up, instantly executes our wfe instruction, and powers down its ALU. The system is now completely silent and frozen.

Software Debugging vs. Hardware Debugging

- **Software Debugging** (VSCode / Standard GDB):
  - **Target:** Applications running inside an OS.
  - **Mechanism:** The OS Kernel provides a system call (e.g., ptrace in Linux). The debugger asks the Kernel to pause the application thread and read its virtual memory. The CPU never actually stops; it is busy running the OS and the debugger.
- **Hardware Debugging** (Bare-Metal):
  - **Target:** The entire System-on-Chip (SoC).
  - **Mechanism:** We use a physical Debug Probe (CMSIS-DAP) connected to a dedicated silicon block inside the CPU (ARM CoreSight). We halt the CPU.

What is GDB again?

- The GNU Debugger (GDB):
  - Created in 1986 by Richard Stallman. The de facto standard debugger for C, C++, and Assembly in systems programming.
- When you click a red dot in VSCode, the editor simply sends a text command (like break main.c:42) to a GDB or LLDB instance running in the background.
- But we'll use the CLI since we can script it, automate it, and manipulate physical memory!

Native vs. Remote Debugging

- **Native Debugging** (Your Laptop):
  - GDB and the target program run on the same machine, under the same OS.
- **Remote Debugging** (Bare-Metal):
  - **Client/Server Model**: GDB runs on your laptop (Client). The code runs on the Pi 5 (Target).
  - GDB communicates using the GDB Remote Serial Protocol (RSP) over a TCP/IP network socket.
  - OpenOCD acts as the "GDB Server". It listens on port 3333, catches GDB's text commands, and translates them into SWD signals.

Refresher: Basic GDB commands

- **(gdb)** prompts indicates the debugger is ready for a command. If the CPU is currently running, you will not see this prompt until the CPU hits a breakpoint or you press Ctrl+C.
- **State Control:**
  - **continue (c)**: Resume execution at full silicon speed (2.4 GHz).
  - **stepi (si)**: Step exactly one physical hardware instruction.
  - **Ctrl+C**: Send an asynchronous interrupt to freeze the CPU instantly.

Refresher: Basic GDB commands

- **Inspection:**
  - **info registers (i r)**: Dump the CPU workspace.
  - **x/<format> <address>**: eXamine memory. (e.g., x/4x 0x80000 = Examine 4 hexadecimal words).
- **Automation:**
  - GDB can run scripts (.gdbinit or **gdb -x script.txt**) to automate complex hardware initialization sequences.

JTAG vs. SWD

- JTAG (**Joint Test Action Group**):
  - An older, industry-standard debugging interface introduced in 1990.
  - Requires at least 4 physical wires (Test Clock, Test Mode Select, Test Data In, Test Data Out).
- SWD (**Serial Wire Debug**):
  - An ARM-proprietary protocol.
  - Optimized for embedded systems. Replaces the 4-wire JTAG with just 2 wires: **SWCLK** (Clock) and **SWDIO** (Bidirectional Data).
- **ARM CoreSight**: The physical debug architecture inside the chip. It bypasses the CPU entirely.

What is OpenOCD?

- OpenOCD (**Open On-Chip Debugger**) is a translation (background-ish) program that runs on your laptop.
- The Hardware Script (cmsis-dap.cfg):
  - **adapter driver cmsis-dap**: Instructs OpenOCD to look for a USB device speaking the ARM CMSIS-DAP protocol (our Debug Probe).
  - **transport select swd**: Instructs the probe to use the 2-wire Serial Wire Debug electrical protocol, rather than 4-wire JTAG.
  - **adapter speed 4000**: Sets the physical clock speed of the SWD wire to 4 MHz.

How does it know about our chip?

```
set _CHIPNAME bcm2712
set _DAP_TAPID 0x4ba00477
set _CHIPCORES 4

swd newdap $_CHIPNAME cpu -expected-id $_DAP_TAPID
dap create $_CHIPNAME.dap -chain-position $_CHIPNAME.cpu
```

- OpenOCD does not auto-discover chip topologies. It doesn't a predefined script for Pi 5 yet. We must explicitly define the BCM2712.
- **DAP** (Debug Access Port): The physical gateway into the ARM CoreSight network.
- **TAPID** (Test Access Port ID): **0x4ba00477**. A hard coded serial number. OpenOCD verifies this immediately to ensure the wires are connected properly.

CoreSight & The Cross-Trigger Interface

```
set _DBGBASE {0x80010000 0x80110000 0x80210000 0x80310000}
set _CTIBASE {0x80020000 0x80120000 0x80220000 0x80320000}

# ... (Inside the Core Loop) ...
cti create $_CTINAME -dap $_CHIPNAME.dap -ap-num 0 -baseaddr [lindex $_CTIBASE $_core]
target create $_TARGETNAME aarch64 -dap $_CHIPNAME.dap -coreid $_core -dbgbase [lindex
$_DBGBASE $_core] -cti $_CTINAME
```

- **DBGBASE:** The memory addresses of the debug registers for Cores 0, 1, 2, and 3.
- **CTI** (Cross-Trigger Interface):
  - A dedicated hardware network connecting all four cores.
  - Allows us to halt Core 0, and have the hardware instantly freeze Cores 1, 2, and 3 simultaneously, preventing race conditions during debugging.
- We explicitly register each core as an aarch64 target, exposing them to GDB.

What's happening in the OpenOCD logs?

```
Info : CMSIS-DAP: Interface ready
Info : clock speed 4000 kHz
Info : SWD DPIDR 0x2ba01477
Info : bcm2712.cpu0: hardware has 6 breakpoints, 4 watchpoints
...
Info : accepting 'gdb' connection on tcp/3333
bcm2712.cpu0 halted in AArch64 state due to debug-request, current mode: EL2H
cpsr: 0x000003c9 pc: 0x80004
```

- **SWD DPIDR 0x2ba01477**: The Debug Port Identification Register.
- CoreSight reports the comparators available for hardware **breakpoints** and **watchpoints**.
- OpenOCD asserts the debug halt signal. The CPU halts. It reports it is running in 64-bit mode, specifically at Exception Level 2 (Hypervisor).
- **pc: 0x80004:** The Program Counter is at the second instruction of our Halt Stub (b _start).

Hardware Security Block

```
Error: Opcode 0xd53e4020, DSCR.ERR=1, DSCR.EL=2
Error: Failed to read ELR_EL3 register
Error: Opcode 0xd53e5200, DSCR.ERR=1, DSCR.EL=2
Error: Failed to read ESR_EL3 register
```

- **TrustZone (EL3):** Exception Level 3 is the Secure Monitor. It isolates cryptographic keys and secure boot processes from the rest of the system.
- The CPU booted our stub in EL2. OpenOCD attempts to read the state of all registers, including EL3 registers. The hardware validates the current privilege level (DSCR.EL=2), determines it lacks Secure clearance, and rejects the read request.

Making the CPU perform addition

- The AArch64 Machine Code:
  - **0xd28000a0: mov x0, #5**
  - **0xd28000e1: mov x1, #7**
  - **0x8b010002: add x2, x0, x1**
- The GDB Commands:
  - **target extended-remote :3333**: Connect to OpenOCD.
  - **info registers**: Fetches the state of the registers.
  - **set {int}<address> = <value>**: Write a 32-bit integer directly to RAM.
  - **set $pc = <address>**: Overwrite the Program Counter.
  - **stepi**: Step Instruction. Cycle the hardware clock exactly one time.

The ARM Philosophy & The Execution State

The RISC Philosophy

- **CISC (Complex Instruction Set Computer)**:
  - x86_64 (Intel/AMD).
  - Instructions vary from 1 to 15 bytes in length.
  - Can perform mathematical operations directly on physical RAM.
- **RISC (Reduced Instruction Set Computer)**:
  - ARMv8-A (Broadcom, Apple Silicon, AWS Graviton).
  - **Fixed Instruction Width:** Every single instruction is exactly 32 bits (4 bytes) long.
  - **The Load/Store Law:** The CPU cannot perform data processing directly on memory.

The AArch64 Execution State

- The '64-bit' designation refers to the data paths.
  - Registers are 64 bits wide.
  - Memory addresses are 64 bits wide.
  - **Instructions are still 32 bits wide.** (faster performance, less program memory)
- Execution State Transitions:
  - The CPU cannot execute AArch64 and AArch32 code simultaneously.
  - State transitions only occur during exceptions or exception returns.

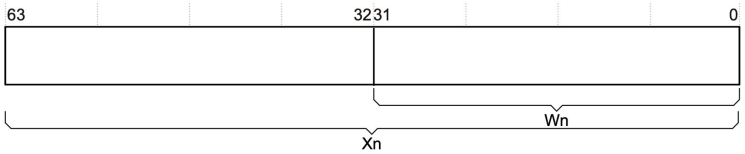
Memory Alignment

- **Alignment:** Accessing data at a memory address that is a multiple of the data's size. (e.g., loading an 8-byte integer from an address ending in 0x0 or 0x8).
- AArch64 hardware is forgiving. Unaligned data loads are permitted but incur a micro-architectural performance penalty (cache line crossing).
- The Stack Pointer (SP) Rule:
  - The hardware enforces strict alignment on the Stack Pointer.
  - The SP MUST ALWAYS be 16-byte aligned whenever it is used to access memory.
  - Failure to align the SP results in an immediate synchronous abort (hardware crash).

The Register File

General Purpose Registers (GPRs)

- 31 General Purpose Registers, physically accessible in one clock cycle.
- 64-bit View (X0 - X30):
  - Hold 64-bit addresses or data.
- 32-bit View (W0 - W30):
  - The lower 32 bits of the exact same physical register.
- The Zero-Extension Rule (Hardware Optimization):
  - Writing to a W register automatically clears the upper 32 bits of the corresponding X register to 0.



	X0/W0
	X1/W1
	X2/W2
	X3/W3
	X4/W4
	X5/W5
	X6/W6
	X7/W7
	X8/W8
	X9/W9
	X10/W10
	X11/W11
	X12/W12
	X13/W13
	X14/W14
	X15/W15
	X16/W16
	X17/W17
	X18/W18
	X19/W19
	X20/W20
	X21/W21
	X22/W22
	X23/W23
	X24/W24
	X25/W25
	X26/W26
	X27/W27
	X28/W28
Frame pointer	X29/W29
Procedure link register	X30/W30

EL0, EL1, EL2, EL3

Zero Register & The Program Counter

- **Register 31 (XZR / WZR): The Zero Register.**
  - Reads always return 0.
  - Writes are instantly discarded (electrically grounded).
  - Purpose: Saves instruction encoding space. (e.g., A dedicated "Compare" instruction is just a "Subtract" instruction where the output is dumped into XZR).
- **The Program Counter (PC):**
  - Contains the 64-bit physical memory address of the current executing instruction.
  - Unlike 32-bit ARMv7, in 64-bit AArch64, the PC is hidden. You can only alter it via explicit branching instructions (B, BL, RET).